

FPGA IMPLEMENTATION OF A CLOCK ENABLED SUPPORTED ADAPTIVE KALMAN FILTERED-BASED SPEED ACCURACY ENHANCEMENT FOR A DC MOTOR

PREFACE

This project, titled “FPGA Implementation of a Clock Enabled Supported Adaptive Kalman Filtered-Based Speed Accuracy Enhancement for a DC Motor” has been prepared within the scope of the graduation project I of Haliç University.

The main purpose of the project is to obtain a more stable and reliable speed information for a DC motor, where low-resolution encoder measurements may become noisy, stepped, and less accurate. In this context, an adaptive Kalman filter approach is considered to combine the system model and measurement data in a more effective way, and the algorithm is planned to be implemented on an FPGA for real time operation.

Throughout this project, we aim to use the theoretical knowledge gained during our courses and transfer it into a practical engineering application. We believe that working on motor modeling, state space representation, adaptive filtering, and FPGA based fixed point design will contribute to our technical development. In addition, the use of a clock enable based architecture is expected to support a more efficient hardware operation by reducing unnecessary computations and power consumption.

Firstly, we would like to thank our advisor for her support during the compilation of this project. Secondly, we thank our lecturers for their valuable contribution towards its compilation. Last but not least, we thank our loved ones for their continued support towards this course. Our greatest wish is that this project will provide a useful perspective for motor speed estimation problems and will contribute, even in a limited way, to applications where accurate and stable speed information is required.

Best Regards,

Emircan İPEK

LIST OF CONTENTS

	Page Number
PREFACE	I
LIST OF CONTENTS	II
LIST OF ABBREVIATIONS	III
LIST OF SYMBOLS	IV
ABSTRACT	1
1.INTRODUCTION.....	2
2. METHODOLOGY	6
2.1 State Space Representation of the DC Motor.....	6
2.2 Kalman Filter	7
2.3 Encoder Measurement	10
2.4 Field Programmable Gate Array (FPGA)	11
2.5 Configurable Logic Block (CLB)	12
2.6 Look-Up Table (LUT)	13
2.7 Multiplexer (MUX)	14
2.8 Fixed-Point Arithmetic and Numerical Stability.....	15
2.9 Clock Enable	15
3. HARDWARE DESCRIPTION	16
4. DESIGNING THE ALGORITHM	19
4.1 Kalman in MATLAB&Simulink.....	19
4.2 FPGA Vivado Design	23
5. FINDINGS AND CONCLUSIONS	26
6.APPENDICES	34
7.CONTRIBUTOR INFORMATION	41
8.REFERENCE	42

LIST OF ABBREVIATIONS

ADC — Analog-to-Digital Converter

CLB — Configurable Logic Block

CPU — Central Processing Unit

DSP — Digital Signal Processor

DC — Direct Current

EKF — Extended Kalman Filter

FF — Flip-Flop

FPGA—Field Programmable Gate Array

KF — Kalman Filter

LUT — Look-Up Table

PWM — Pulse Width Modulation

LIST OF SYMBOLS

$x(k)$ — State vector (discrete-time)

$\hat{x}(k)$ — Estimated state vector

$u(k)$ — System input (control input)

$y(k)$ — System output / measurement (encoder-based speed measurement)

A — State matrix

B — Input matrix

C — Output matrix

D — Feedthrough matrix

Q — Process noise covariance matrix

R — Measurement noise covariance matrix

$P(k)$ — Error covariance matrix

$K(k)$ — Kalman gain

ω — Angular speed (motor speed)

$\hat{\omega}$ — Estimated angular speed

i_a — Armature current

V_a — Armature voltage

R_a — Armature resistance

L_a — Armature inductance

K_e — Back-EMF constant

K_t — Torque constant

J — Rotor inertia

B_m — Viscous friction coefficient

T_L — Load torque

T_s — Sampling time

$e(k)$ — Residual / estimation error ($y(k) - \hat{y}(k)$)

FPGA IMPLEMENTATION OF A CLOCK ENABLED SUPPORTED ADAPTIVE KALMAN FILTERED-BASED SPEED ACCURACY ENHANCEMENT FOR A DC MOTOR

ABSTRACT

This project aims to improve the speed of a DC motor using a low-resolution encoder in a more accurate and stable way. Encoder measurements can often be discrete and noisy. This situation makes it difficult to use the speed information directly and can negatively affect control performance.

For this reason, an adaptive Kalman filter approach that can reduce measurement noise and evaluate the measurement together with model information will be used in this project.

First, the electrical and mechanical model of the DC motor will be enhancement. Then, this model will be expressed in state space form and converted to discrete time for numerical implementation. In the MATLAB framework, it is planned that the Kalman filter will be set up with current and voltage measurements and system model in hand. The R matrix, or measurement covariance of the filter, will be managed so it is adjusted with regard to certain working conditions.

To demonstrate the algorithm's hardware applicability, the Kalman filter will be adapted to fixed-point arithmetic. It will then be designed to run in real time on an FPGA. To reduce power consumption, a clock enables based structure that operates only during sampling intervals will be used.

In the final steps, the speed estimate obtained on the FPGA will be compared with the low-resolution encoder measurement, and the accuracy and stability performance will be analyzed.

Keywords: DC motor speed estimation, Adaptive Kalman filter, FPGA, Fixed-point arithmetic, Clock enable, Measurement noise covariance (R).

1.INTRODUCTION

Nowadays, obtaining accurate and stable speed information in the control of electric motors is considered a fundamental requirement for evaluating system performance [3]. It is known that in DC motors used in industrial applications, unreliable speed feedback reduces control accuracy and negatively affects system stability [3]. Therefore, measuring and analyzing motor speed has an important place in the design of control systems [2], [3].

In practice, speed information is mostly obtained through increment low-resolution encoders. However, when encoders are used, it has been observed that measurement data shows a stepped, vibrating, and noisy structure [5]. This situation causes differences between the measured speed and the actual dynamic behavior of the motor, and these differences can lead to incorrect decisions in feedback control systems [5]. Under these working conditions, where measurement noise is significant, relying on speed information based on sensor data alone can result in undesirable oscillations and performance losses [5].

Within the scope of this graduation project, an adaptive Kalman filter based speed estimation method is addressed to improve the speed information of a DC motor using an encoder. Assuming that measurement noise varies depending on the motor speed, the goal to update the measurement noise covariance used in the Kalman filter, the R matrix, based on the instantaneous measurement error [9].

Within the scope of the project, a mathematical model representing the electrical and mechanical behavior of the DC motor was first enhancement, and the obtained model was expressed in state space form [3], [4].

The continuous time system model was converted to discrete time to suit numerical applications, thus preparing the necessary structure for speed estimation [8]. At this point, a Kalman filter structure was used, which allows for the joint evaluation of the system model and the measurement data obtained from the encoder [1], [4].

The developed speed estimation method was not limited to evaluation in a software environment; the implementation of the algorithm at the hardware level was also a target. Kalman filter calculations were performed using MATLAB. In this line, the Kalman filter algorithm was adapted to fixed-point arithmetic and designed to run in real time on an FPGA [7],[13].

The reason for using fixed point arithmetic is to the usage of hardware sources such as LUT, FF, and prevent hardware complexity that least to lower cost [7],[13].

In the hardware implementation, a clock enables based control structure was preferred to ensure that the calculation units are active only at sampling times [6]. The clock enable is embedded in the FF in the LUTs within the FPGA by doing so, the activation of speed estimation at sampling times is realized. This is done with the view to avoid unnecessary computations and utilizing hardware resources more effectively to reduce power consumption [6],[7],[13],[14]. The developed method will be then tested in the experimental part, where the accuracy of estimated speed values provided from the FPGA will be considered with the real speed measures given by the low-resolution encoder. The results obtained aim to demonstrate that the adaptive structure is effective in reducing measurement noise [5], [9].

Accurate speed measurement of DC motors has long been considered a research topic in motor control and performance analysis works [3]. The literature shows that different solutions have been suggested to overcome the noise and uncertainty problems associated with the use of direct sensor data in speed measurement [2], [3]. It is reported that estimation methods based on the joint consideration of the system model and measurement data are preferred in a significant part of these approaches [1], [4].

Various researches indicate that in DC motor systems using encoders, measurement accuracy decreases [5].

It is noted that this situation can lead to undesirable results such as oscillation, delay, and unstable behavior in feedback control systems [3], [5]. It is stated that low-pass filters and digital filtering methods can be used to reduce measurement noise however, these approaches can cause delays in system dynamics and limit real time control performance [3].

Among model-based estimation methods, the Kalman filter is widely used in the literature because it can statistically handle measurement noise [1], [4]. In Kalman filter based research, it is stated that speed estimation can be performed by enhancement mathematical models representing the electrical and mechanical behavior of DC motors [3], [4]. A significant part of these research reports that the Kalman filter is applied with fixed process and measurement noise covariances; however, it is reported that fixed covariance values can limit filter performance under variable operating conditions, it may not always be possible to achieve sufficient accuracy [9]. It is stated that adaptive Kalman filter approaches are gaining increasing attention in the literature to overcome these limitations [9].

In adaptive structures, it is noted that the measurement noise covariance is updated based on the systems behavior at any given moment, enabling the filter gain to be better adjusted [9]. Different researches show that adaptation methods based on the magnitude of measurement error residual signal can increase the accuracy of speed estimation [9]. However, it is noted that hardware-based applications of adaptive Kalman filter approaches are limited in terms of number and that application cases are mostly concentrated in software environments [7],[12],[13].

In the analysis of FPGA based implementation of the Kalman filter algorithm, it is understood that many advantages can be achieved with regards to real time systems by taking into account the parallel processing potential of FPGAs [6]. In contrast, with regards to the use of FPGA devices with a small number of resources available, it has been noted by many researchers that a preference towards fixed point arithmetic with the approach of a simplified algorithm is preferred [7],[13]. Although, it has been noted with regards to a wide range of FPGA applications that a special approach to power efficiency optimization is not fully considered [6] Although, techniques such as a clock enable approach are noted by some researchers, it is thought that this approach is not considered with a complete framework with regards to speed estimation algorithms involving a Kalman filter [6],[14].

In this context, when the studies examined are evaluated together, it is seen that studies on speed estimation with adaptive Kalman filters in DC motors using encoders and the application of this approach on FPGAs with power efficiency in mind are relatively limited in the literature [5], [6], [7],[9],[12],[13]. This situation is thought to point to a research gap that requires the joint evaluation of adaptive estimation methods and hardware implementations [7],[9],[12],[13]. In this project, we aimed to research the gap in the literature by focusing on this topic, which we observed to be relatively limited.

As a result, this graduation project addresses the DC motor speed estimation problem within the framework of an adaptive Kalman filter, fixed-point hardware design, and FPGA based real time application [1], [6], [7], [9]. It is considered that the presented approach can help obtain more reliable speed information with low-cost sensors and offers a suitable solution for motor control systems [5], [9]. Then the FPGA based speed estimator developed will be tested on a real DC motor, and the Kalman filter estimation performance will be validated against the raw encoder data to evaluate the method's ability to filter measurement noise while maintaining accuracy and stability [5], [6], [7],[13]."

2. METHODOLOGY

This part how to do things for the project. The method for the project is explained here. First it says why certain computer parts and methods were chosen. Then it gives a summary of how the DC motor works and the idea behind using the Kalman filter to figure out how fast the motor is going. The DC motor and the Kalman filter are important, for the project.

2.1 State Space Representation of the DC Motor

To use a Kalman filter to estimate you first have to describe the system with math. For a D.C. motor this means writing differential equations . These take into account its electrical parts (resistance, inductance and back E.M.F.) and its mechanical parts (inertia, friction and load torque).

The equations are then transformed into state-space form. In this format, the system state vector (what we want to estimate), any inputs, and the outputs are clearly presented. The state-space approach provides a convenient way to characterize the evolution of a system, and for control and estimation. One of the educational benefits of the state-space model is that it makes clear what drives the system. For instance, if you want to estimate speed of a DC motor, you would naturally include angular speed in your state vector. But speed may not be sufficient in itself. Besides the mechanical states, electrical states such as current are added to the state vector for better modeling of the torque generation and back EMF effects. This improves the model used in the prediction step of the Kalman filter. Also external forces such as load torque can be treated as "process noise". This means that even if they are not explicitly written into the main equations, they can be taken into account by the Q matrix. a DC motor, you would naturally have angular speed in your state vector. However, speed alone may not be enough. Besides the mechanical states, electrical states such as current are added to the state vector for better modeling of the torque generation and back EMF effects. This enables the model used in the prediction step of the Kalman filter to be more accurate. In addition, the effect of external forces like load torque can be considered as "process noise". This means that even if they are not explicitly written in the main equations, they can be accounted for using the Q matrix [3],[4].

$$x = \begin{bmatrix} \omega \\ i_a \end{bmatrix}, u = V_a, y = \omega$$

$$\frac{dx}{dt} = A x + B u, y = Cx$$

$$J \frac{d\omega}{dt} = K_t i_a - B\omega + t_l$$

$$L_a \frac{di_a}{dt} = -R_a i_a - K_e \omega + V_a$$

$$A = \begin{bmatrix} -\frac{B}{J} & \frac{K_t}{J} \\ -\frac{K_e}{L_a} & -\frac{R_a}{L_a} \end{bmatrix}, B = \begin{bmatrix} \omega \\ i_a \end{bmatrix}, C = [1 \ 0]$$

2.2 Kalman Filter

The Kalman filter is designed to estimate system states under noisy measurements using the minimum error variance approach. Its basic operation can be explained in prediction and update. In the prediction step, the possible value of the next state is estimated using the system model, while in the update step the measurement and the prediction are combined to obtain a more confident estimate [1], [4]. Regular iteration of these two steps makes it possible to produce a “smoother” and “closer” state estimate over time, even if the measurement noise is high [1].

The instructive aspect of the Kalman filter is that it clearly shows the balance of confidence between the measurement and the model. This balance is based on the Kalman gain: if the measurement noise is high, less confidence is placed in the measurement, and the model weight increases; if the measurement is more reliable, the measurement weight increases [2], [4]. The Kalman Filter works due to the existence of both process noise covariance Q, and measurement noise covariance R. Generally, the process noise covariance Q will show you how confident the Kalman Filter Model is in knowing what it knows, and how many external factors it will be able to consider. Process noise covariance R, on the other hand, shows you the potential for error that exists when your sensors measure. In addition to making your signal clearer, the Kalman Filter is also a method for dealing with uncertainty within your system.

It is therefore considered a fundamental tool in control and estimation literature [2], [3], [4].

In addition, since the filter equations are matrix-based, they are considered together with additional engineering decisions such as fixed-point conversion and resource planning in hardware implementations [7],[13].

The method to be followed in this study includes obtaining the mathematical model of the DC motor, converting the model to discrete time, designing the Kalman filter, and implementing the designed filter in real time both in the MATLAB environment and on the FPGA. First, a state space model representing the electrical and mechanical dynamics of the motor was obtained [8].

The reason for choosing FPGA is to meet the real time speed estimation requirement due to its parallel processing capabilities and to achieve low power consumed with fixed point arithmetic.

For the Kalman filter, the process noise covariance $Q = 10^{-4} I$ and the measurement noise covariance $R = 10^{-3}$ were initially selected. However, since measurement noise increases as speed decreases in low-resolution encoders, an adaptive R update will be used in this project as an approach that is missing in the literature [9]. Thus, the Kalman filter will adjust itself in real time as the measurement uncertainty changes.

The filter will be implemented using the following equations:

$$x_{k|k-1} = A_d x_{k-1|k-1} + B_d u_k$$

$$P_{k|k-1} = A_d P_{k-1|k-1} A_d^T + Q$$

$$K_k = P_{k-1|k-1} C^T (C P_{k-1|k-1} C^T + R_k)^{-1}$$

$$X_{k|k} = X_{k|k-1} + K_k (y_k - C x_{k|k-1})$$

$$R_k = R_0 + \alpha |y_k - C x_{k|k-1}|$$

State Estimation Equations

$x_{k|k-1}$: Estimated State Vector

$x_{k-1|k-1}$: Previous Condition Estimate

A_d : System Matrix (Discrete-Time)

B_d : Input Matrix

u_k : Control Input

Error Covariance Estimation

$P_{k|k-1}$: Estimated Error Covariance Matrix

A_d^T : Transpose of the System Matrix

Q : System Noise Covariance

$P_{k-1|k-1}$: Previously Updated Error Covariance Matrix

Kalman Gain

K_k : Kalman Gain

C : Measurement Matrix

R_k : Measurement Noise Covariance

State Update (Correction) Equation

$X_{k|k}$: Updated (Most Accurate) Status Estimate

y_k : Measured Value

Adaptive Updates (Updating the R)

R_k : Current Measurement Noise

R_0 : Initial Fixed Noise Value

α : Adaptation Coefficient

Here, α is an experimentally determined adaptation coefficient. In the second step of the design, the Kalman filter will be converted to fixed-point arithmetic and implemented as hardware on the FPGA [7],[13].

A clock enables based control architecture will be used to ensure that the filters calculation steps only run at the sampling instant, thus optimizing power and FPGA resource usage. In the last stage, the estimated speed values obtained from the MATLAB simulation will be compared with the real time speed estimations obtained from the FPGA to check the system. The real-time speed estimation algorithm is based on the estimation and update equations given in Figure 1.1. In this model, the

measurement noise (R) is dynamically updated, independent of system noise.

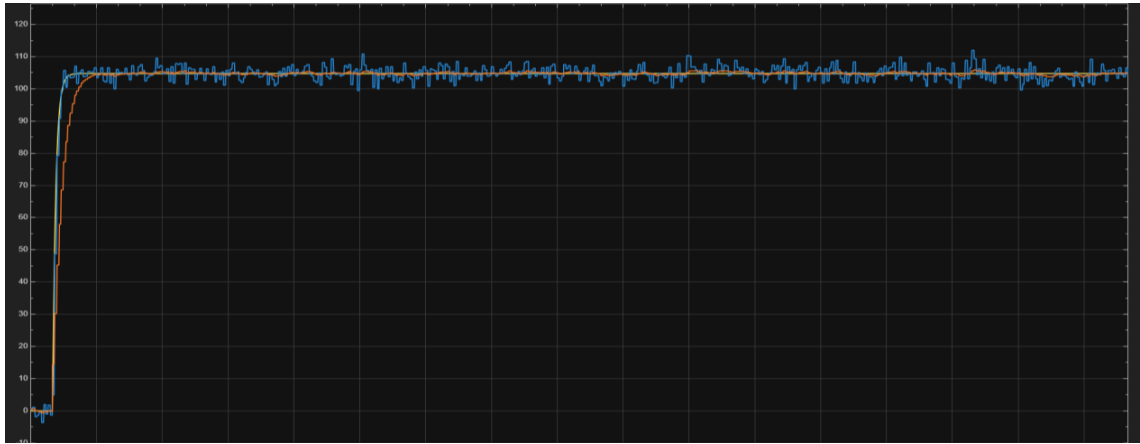


Figure 1.1 Adaptive R with Kalman filter-based Speed Estimation

2.3 Encoder Measurement

In encoders with an incremental type, normally the speed information is generated by counting the number of pulses within a particular time interval. When the feedback signal in control systems is taken directly from this measurement, unnecessary corrections, oscillations, or instability may be observed [5].

Measurement noise is not only electronic noise the sampling period, counting method, edge detection errors, and mechanical vibrations can also affect the measurement. For this reason, encoder measurement may show different error characteristics depending on the speed range. The literature indicates that a smoother and more reliable speed signal can be obtained using Kalman filtering in DC motors with encoders [5]. The logic here is that if the measurement alone is not reliable enough, it can be combined with the system model to produce a more stable estimation [1], [4].

2.4 Field Programmable Gate Array (FPGA)

FPGA is a digital platform that can be reconfigured at the hardware level. While operations in general purpose processors are mostly executed sequentially FPGAs enable multiple operations to be executed in parallel at the same time. This feature can provide advantages in terms of time determinism especially in estimation algorithms such as Kalman filters that involve matrix operations repeated throughout the sampling period. This is because in FPGA design, which operation will be completed in how many clock cycles is planned in a more predictable way, which is important in real-time control and estimation applications [6].

When choosing an FPGA, not only speed but also hardware resource management is considered. For example, resources like LUT, FF, DSP are directly related to the level of parallelism in the design, higher parallelism consumes more resources, while more sequential designs reduce resource usage but increase control logic. This balance is particularly critical in Kalman filter type structures as the algorithm involves numerous multiplication and addition steps [7],[13]. In addition, when fixed point arithmetic is implemented on an FPGA power consumption and area cost generally become more manageable. For this reason, the fundamental reasons for the use of FPGAs in motor driver control and prediction-based applications, as documented in the literature can be summarized as the need for real-time operation, the advantage of parallel calculation and suitability for fixed-point implementation [6], [7],[13].

2.5 Configurable Logic Block (CLB)

The basic components of an FPGA are CLBs. Within a CLB, there are supporting structures such as LUTs, flip-flops, and in most architectures, multiplexers and carry chains. This structure allows the algorithm steps to be set up in a hardware topology rather than being executed sequentially like software commands. This means that the computation steps are performed on signals moving along a specific data path, enabling timing analysis [6].

The importance of CLBs comes from their direct impact on the “scalability” of the design. For example, in a Kalman filter, many intermediate variables are generated, such as matrix updates, gain calculations, and error covariance steps. FFs are used to store these intermediate variables, while LUTs are used for logic decisions and simple combinational operations. Effective planning of resources within the CLB determines both the speed and resource consumption of the design. Also, if the same hardware block is to be used in different steps in some implementations, the combination of MUX, FF within the CLB plays a critical role; that is, the CLB is considered not just a “small circuit piece” but the main building block that forms the hardware map of the algorithm [6]. For this reason, in FPGA based Kalman applications, CLB distribution and routing load are considered important design parameters in terms of timing closed loop [6].

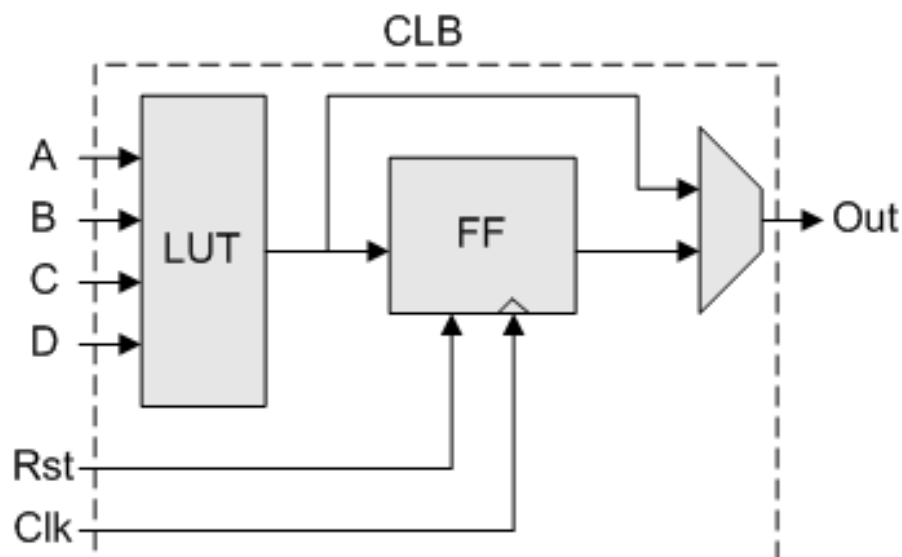


Figure 1.2 CLB Structure

2.6 Look-Up Table (LUT)

The LUT, the basic unit of the FPGA architecture, is a hardware structure that produces outputs based on input signals. Basically, it is a programmable truth table that can be customized according to the desired logic function. A LUT operates based on a truth table logic that produces an output corresponding to input combinations. It provides flexibility because logical functions can be defined based on the LUT content; that is, different functions can be implemented on the same hardware area using different LUT contents. In this way, the LUT is fundamental to the reprogrammable nature of an FPGA. However, the concept of LUT should not be limited to logic function generation. In hardware design, LUT is also considered as a table from which predefined values are selected and this second use can provide practical advantages in estimation algorithms [7],[13]. For example, when measurement uncertainty is assumed to vary according to speed ranges, it may be desirable to place values related to measurement R in a range-based table. Thus, instead of generating R with complex statistical calculations for each sampling, the appropriate R range is selected according to the magnitude of the measurement error. This method can help reduce steps such as division, complex multiplication, which have high processing costs on the FPGA [7],[13]. Also, since the need for constraints for stability is emphasized in the adaptive R approach, LUT based selections are practical in terms of applying safety limits such as the “min max range” [9].

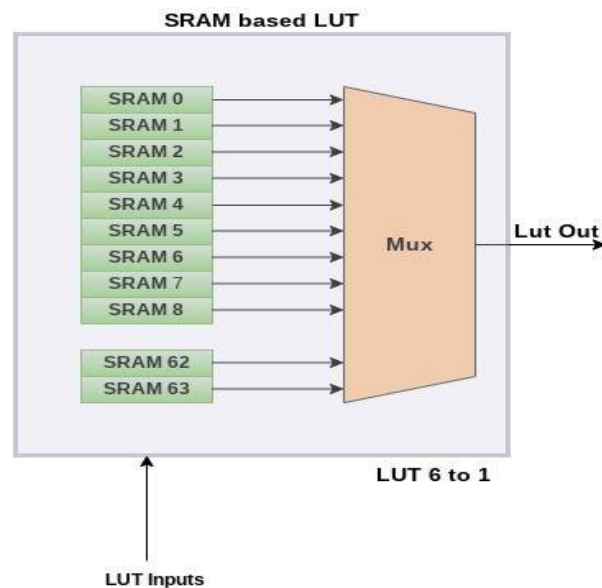


Figure 1.3 LUT Structure

2.7 Multiplexer (MUX)

A MUX is a switching element that allows one of multiple inputs to be selected and transferred to the output. Although it may appear to be a simple selection circuit at first view, MUXs are key elements in data path design that determine the flow of the design. Especially when resource sharing is desired on an FPGA, the same arithmetic block must be fed with different signals at different time steps this feeding is usually performed via a MUX [6].

Structures such as Kalman filters contain many intermediate variables, state estimates, measurement estimates, residuals, covariance updates, gain matrices, etc. Some of these variables can be processed in the same multiply and add unit, thus, fewer DSP blocks or LUT based arithmetic units are required, reducing area and power requirements. However, the cost of this advantage is an increase in the number of MUXs required to manage the data flow and the need for effective control signal planning. Control signals are often generated by a state machine (FSM), which determines which data to select at each step [6]. As the number of MUXs increases, routing load and delay may increase, so care must be taken in terms of timing closing. Therefore, MUX usage is seen as a practical way to balance speed, resources, and complications more parallelism may require fewer MUXs but consumes more resources; more sharing works with fewer resources but increases the need for control MUX [6].

In this context, MUX holds an important place in FPGA based estimation systems.

2.8 Fixed Point Arithmetic and Numerical Stability

The primary aim of fixed-point representation is that of decreasing the cost of realization with the use of bit representations of a certain length. Even though floating point computation can provide sufficient range and accuracy, they can increase resource usage on FPGAs. Herein lies the reason why fixed point representation is preferable for the realization of matrix algorithms such as the Kalman filter [7], [13]. However, in case of fixed point computations, scaling and quantization are not required; therefore, it is essential to take into account both precision and resource utilization at once [4]. One of the key elements in this regard is the correct determination of the word length and fractional length of variables. Since the state vector and covariance matrix may not have the same dynamic range, using a single format everywhere can increase the risk of overflow. In practice, error buildup is reduced by using wider bits in intermediate calculations, followed by cutting and rounding [7],[13]. This issue becomes even more important in adaptive structures because updating covariance values such as R changes the Kalman gain and affects how much weight the system gives to estimation modeling. If quantization errors grow with the gain, oscillations or delays may be observed in the estimation [9]. Therefore, in fixed-point implementations, only “arithmetic conversion” is not performed [4], [7], [9],[13].

2.9 Clock Enable

Clock enable is a control approach that ensures register updates are performed only under specific conditions. In sampling-based algorithms, calculations such as the Kalman filter are desired to be performed at sampling times rather than every clock cycle. On an FPGA, the clock signal typically operates at a high frequency; in contrast, Kalman filter updates may be tied to a lower sampling period T_s . In this case, unnecessary clock cycle switching of registers increases dynamic power consumption. Using clock enable can reduce power consumption by preventing unnecessary register updates [14]. Clock enable also simplifies the step-by-step control logic in sequential computation architectures. Control of these updates via an enable signal protects data compatibility. However, the enable signal is typically generated by an FSM, and incorrect timing can lead to problems such as metastability, data skipping, or skipping steps; as a result, enable timing is carefully considered in design verification [14].

3. HARDWARE DESCRIPTION

The objective of this project is to measure the speed of a DC motor using encoder data and to obtain a more stable speed estimate using a Kalman filter. The system is designed to operate in real time on an FPGA. For this reason, every hardware component used was selected to be compatible with both the control system and the FPGA. The project mainly uses a 12 V DC motor with an encoder, a TB6612FNG motor driver module a Digilent Basys 3 FPGA development board, a 12 V power supply, a breadboard, jumper wires and the Vivado ILA debug framework.

The Digilent Basys 3 FPGA development board is the part that controls and does the computing for this project. This Basys 3 board has an FPGA from the Xilinx Artix-7 family. What is really good about the FPGA is that it can do lots of things at the time and it can do them in hardware, which is different from the usual microcontrollers. This makes the Digilent Basys 3 FPGA development board really good, for projects that need to control motors in time and figure out how fast they are going. In this project the Digilent Basys 3 FPGA development board generates PWM signals reads the encoder signals calculates the speed and runs the Kalman filter algorithm by itself.

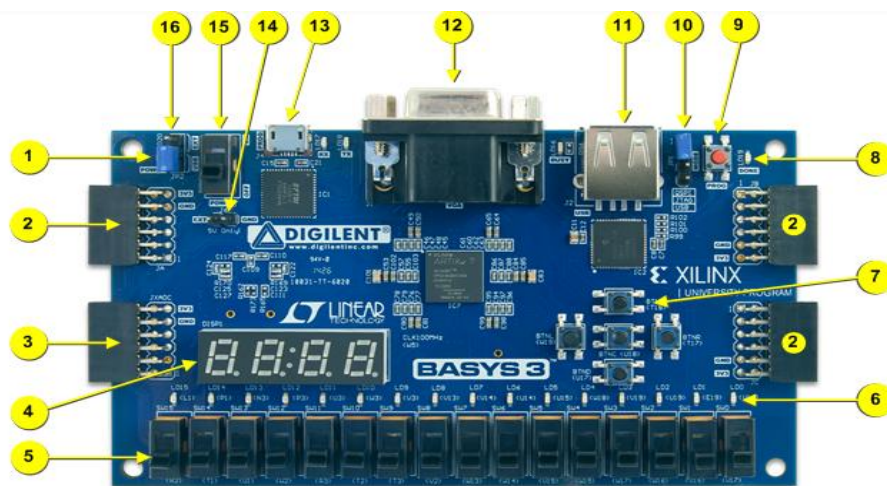


Figure 1.4

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod connector(s)	10	Programming mode jumper
3	Analog signal Pmod connector (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

Basys3: Pmod Pin-Out Diagram

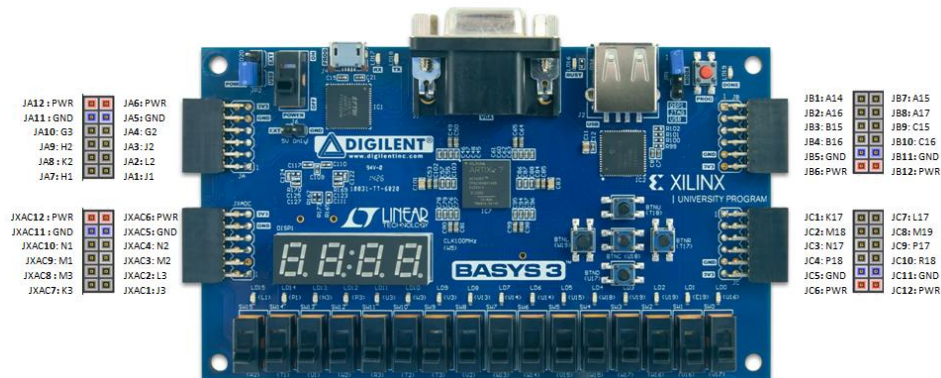


Figure 1.5

A 12 volt DC motor with a 1000 RPM encoder was used in this project. The main reason I chose this motor was that it came with an encoder. This encoder helped measure the motors speed using pulses. The data from these pulses was then compared with the output from a Kalman filter. By doing this I could analyze both the speed from the encoder and the estimated speed from the Kalman filter in the same system. The encoder was really useful, for getting speed readings. It provided a way to validate the Kalman filters performance. The motor and encoder worked well together for this project.



Figure 1.5.1

The TB6612FNG motor driver was very useful in driving the DC motor. We encountered an issue in that the FPGA pins did not have enough voltage to drive the DC motor on their own. Hence, the motor driver circuit took on the role of a man in the middle, between the FPGA and the DC motor. The FPGA would send signals to the motor driver which would dictate how fast and in which direction the DC motor would operate.

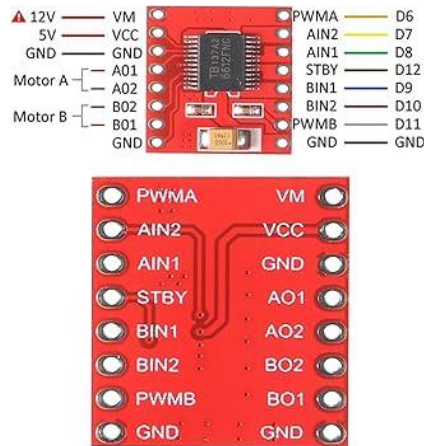


Figure 1.5.2

For supplying the motor with the required voltage, a Mervesan MRW-60-12-S 12V 5A power supply was used. Powering the FPGA board is through USB cable. However, the power for the motors is supplied separately from that of the FPGA board. This helps ensure that the FPGA board remains safe from the motor's current.

A common GND connection was made between the FPGA, motor driver, encoder and power supply to ensure the system works correctly. The circuit connections were made using a breadboard and jumper wires. The TB6612FNG motor driver was put on the breadboard. The motor, encoder, power supply and Basys 3 Pmod pins were connected using these points. This setup made it easy to check the connections and make changes during testing. The switches and LEDs on the Basys 3 board were used during testing. The switches controlled the PWM duty cycle value, the motor direction and the encoder/Kalman output selection.

The LEDs showed the speed value in binary format. This way the motor speed could be monitored using the LEDs on the FPGA and, through Vivado ILA. The motor speed was checked on the LEDs. The Basys 3 boards switches and LEDs helped a lot during testing.

4. DESIGNING THE ALGORITHM

4.1 KALMAN DESIGN IN MATLAB & SIMULINK

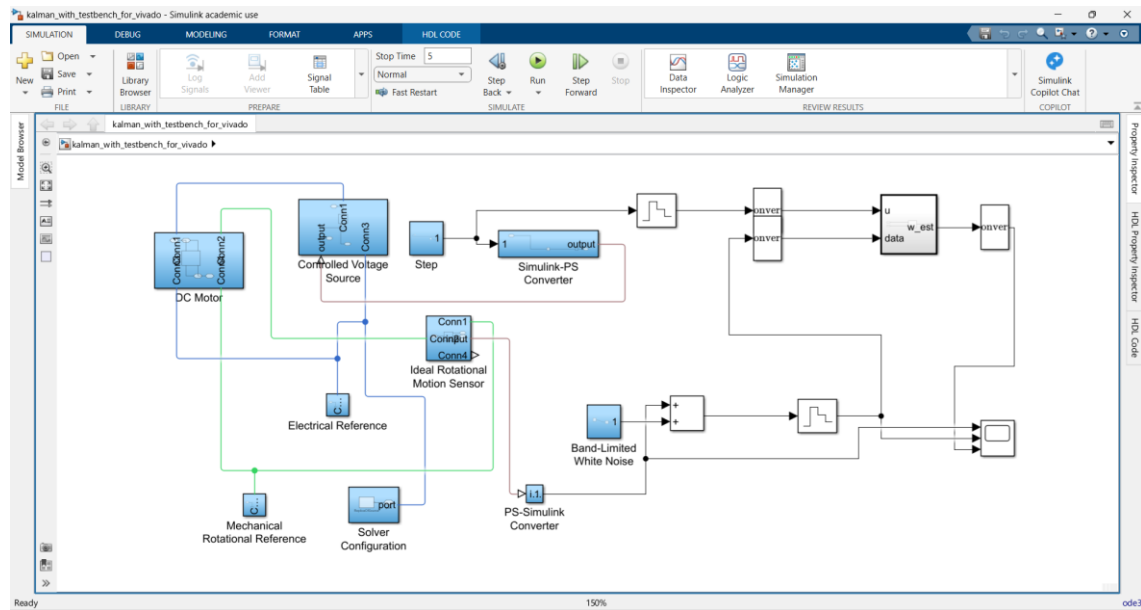


Figure 1.6

Figure 1.6 shows the high-level Simulink model developed to validate the DC motor speed estimation algorithm prior to hardware implementation. The model combines the Kalman filter subsystem tested using a Simscape-based physical model representing a Robotzade 12V, 1000 RPM DC motor with the high-level Simulink model. A Step block provides the 12V armature voltage command; this signal is transferred to the Simscape physical domain used by the DC Motor block via the Controlled Voltage Source and Simulink PS Converter blocks. An Ideal Rotational Motion Sensor block measures the rotor's mechanical angular velocity, and this value is converted back to the Simulink signal domain using the PS-Simulink Converter block. To simulate the quantization and mechanical vibration noise generated in practice by a real, low-resolution incremental encoder, Band-Limited White Noise is intentionally added to the measured speed signal. The resulting noisy signal is fed into the Kalman filter subsystem along with the same voltage command used to drive the motor. The Simscape network needs a few blocks to work properly. These blocks are the Electrical Reference, Mechanical Rotational Reference and Solver Configuration. They help set up the network infrastructure. The Scope block is used to compare two speed signals in time.

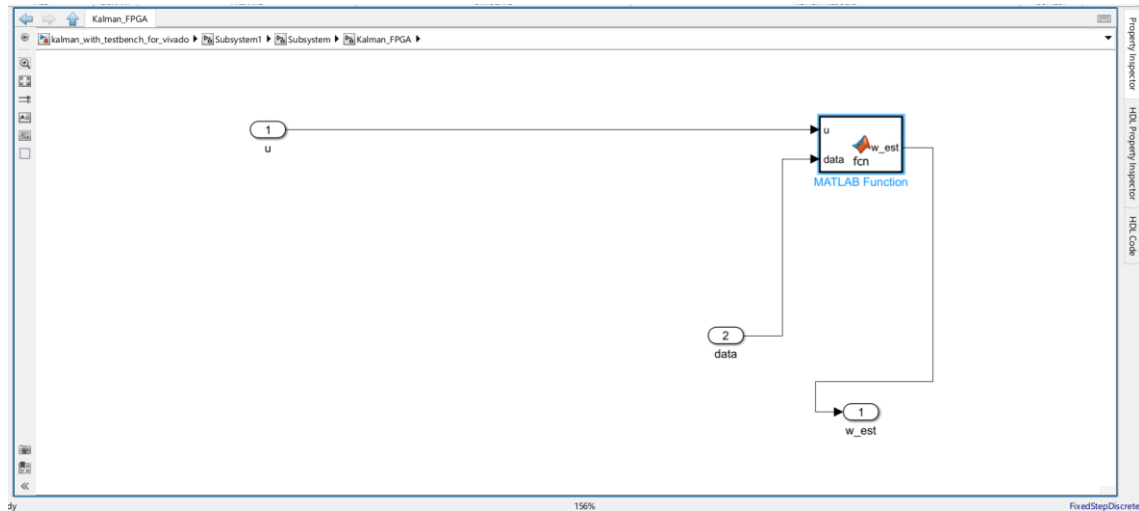


Figure 1.7

Figure 1.7 shows the internal structure of the Kalman filter subsystem referenced in the high level model shown in Figure 1.6. The subsystem has two inputs: the commanded armature voltage “u” and the noisy measured speed “data”. Each of these signals passes through a data type conversion block before reaching the core MATLAB Function block and output is w_est.

Annotated MATLAB/Simulink Kalman Filter Code

```

1 function w_est = fcn(u, data)
2 %
3 % Real-Time Adaptive Fixed-Point Kalman Filter for DC Motor Speed Estimation
4 %
5 % Inputs:
6 % u : Motor control voltage (calculated from duty cycle & direction in Q16)
7 % data : Measured motor speed from the Encoder Reader module in Q16
8 % Output:
9 % w_est: Optimal, noise-filtered motor speed estimation
10 %
11
12 % 1. Fixed-Point Data Type Definition (Q16.16 Format)
13 % NumericType(Signed=1, WordLength=32, FractionLength=16)
14 % Why: 32-bit width provides a massive dynamic range, while 16 fractional bits
15 % guarantee high numerical precision. This fits perfectly into FPGA DSP slices.
16 T = numericType(1, 32, 16);
17
18 % 2. Fixed-Point Arithmetic Rules
19 % Why: Truncation ('Zero') saves gate area on FPGA compared to nearest-rounding.
20 % Saturation ('Saturate') prevents numerical wrapping (e.g., positive turning
21 % into negative) which would cause catastrophic motor control failures.
22 F = fimath('RoundingMethod', 'Zero', 'OverflowAction', 'Saturate');
23
24 % 3. Persistent Variables (Registers)
25 % Why: These variables must retain their values between sample steps (acting
26 % as registers/memory inside the FPGA design).
27 persistent x_est P R_k Ad Bd Cd Q alpha R0 Ad_t Cd_t I initialized
28
29 % 4. System Initialization (Executed only once on Startup/Reset)
30 if isempty(initialized)
31
32     % Discrete State-Space System Matrices (Ad, Bd, Cd)
33     % x[k+1] = Ad*x[k] + Bd*u[k]
34     % y[k] = Cd*x[k]
35     % State Vector x = [Speed; Acceleration/Load Bias]
36     Ad = fi([0.8500, 0.2000; -0.0100, 0.4000], T, F); % State transition matrix
37     Bd = fi([1.3050; 0.1000], T, F); % Input distribution vector
38     Cd = fi([1, 0], T, F); % Only State 1 (Speed) is measured
39
40     % Transposed Matrices for Covariance Math
41     Ad_t = fi([0.8500, -0.0100; 0.2000, 0.4000], T, F); % Ad Transposed
42     Cd_t = fi([1; 0], T, F); % Cd Transposed
43     I = fi([1 0; 0 1], T, F); % Identity Matrix (2x2)
44
45     % Initial States & Covariances
46     x_est = fi([0; 0], T, F); % Start with motor at 0 speed
47
48     % P Matrix: Initial Estimation Error Covariance (High uncertainty at start)
49     P = fi([5.0, 0; 0, 20.0], T, F);
50
51     % Q Matrix: Process Noise Covariance (How much we trust our motor model)
52     Q = fi([0.2, 0; 0, 0.2], T, F);

```

```

53
54 % Adaptive Measurement Noise Parameters
55 R0 = fi(5.0, T, F); % Base measurement noise covariance
56 alpha = fi(0.02, T, F); % Scaling factor for adaptation
57 R_k = fi(5.0, T, F); % Initial measurement noise covariance
58
59 initialized = 1; % Set initialization flag to True
60 end
61
62
63 % =====
64 % KALMAN FILTER ALGORITHM STEP-BY-STEP
65 % =====
66
67 % STEP 1: PREDICT PHASE (ZAMANDA İLERLETME)
68 % Why: Predicts the next state (x_pred) based on the current motor state (x_est)
69 % and the applied control voltage (u).
70 x_pred = fi(Ad * x_est + Bd * u, T, F);
71
72 % STEP 2: ERROR COVARIANCE PREDICTION
73 % Why: Estimates how much error/uncertainty is in our prediction.
74 % Equation: P_pred = Ad * P * Ad' + Q
75 P_pred = fi(Ad * P * Ad_t + Q, T, F);
76
77
78 % STEP 3: MEASUREMENT PREDICTION
79 % Why: Predicts what speed the encoder should read based on our state prediction.
80 y_pred = fi(Cd * x_pred, T, F);
81
82
83 % STEP 4: CALCULATE RESIDUAL (MEASUREMENT ERROR)
84 % Why: Measures the difference between actual encoder reading and predicted speed.
85 residual = fi(data - y_pred, T, F);
86
87
88 % STEP 5: ADAPTIVE MEASUREMENT COVARIANCE UPDATE (R_k)
89 % Why (IMPORTANT JURY INFO): This is a brilliant feature! If there is high
90 % high-frequency noise or a huge mechanical vibration, 'residual' spikes up.
91 % This formula dynamically INCREASES R_k. When R_k is high, the filter ignores
92 % the noisy encoder and relies MORE on the motor mathematical model to keep
93 % the output speed steady.
94 R_k = fi(R0 + fi(alpha * abs(residual), T, F), T, F);
95
96
97 % STEP 6: BOUNDS FOR ADAPTIVE R_k
98 % Why: Prevents R_k from becoming too small (which would bypass the filter)
99 % or too large (which would ignore physical measurements indefinitely).
100 if R_k < fi(0.1, T, F)
101     R_k = fi(0.1, T, F);
102 elseif R_k > fi(20.0, T, F)
103     R_k = fi(20.0, T, F);
104 end
105
106
107 % STEP 7: INNOVATION COVARIANCE (S_k)
108 % Why: Prepares the denominator term for the Kalman Gain calculation.
109 S_k = fi(fi(Cd * P_pred * Cd_t, T, F) + R_k, T, F);
110
111
112 % STEP 8: KALMAN GAIN CALCULATION WITH DIVISION SAFEGUARD
113 % Why: Computes the optimal correction weight (K).
114 % Guard: If S_k is extremely small, we bypass division to prevent division-by-zero
115 % errors inside the FPGA DSP blocks, which would freeze the hardware logic.
116 if abs(S_k) < fi(0.001, T, F)
117     K = fi([0; 0], T, F); % Do not update if denominator is zero/invalid
118 else
119     K = fi((P_pred * Cd_t) / S_k, T, F); % Standard Kalman Gain formula
120 end
121
122
123 % STEP 9: STATE CORRECTION (GÜNCELLEME)
124 % Why: Combines the prediction and the noisy measurement using Kalman Gain (K).
125 % This yields the optimal, filtered speed and bias estimation.
126 x_est = fi(x_pred + fi(K * residual, T, F), T, F);
127
128
129 % STEP 10: ERROR COVARIANCE GÜNCELLEME
130 % Why: Updates the uncertainty matrix P for the next step.
131 % Equation: P = (I - K*Cd) * P_pred
132 P = fi(fi(I - fi(K * Cd, T, F), T, F) * P_pred, T, F);
133
134
135 % STEP 11: SYMMETRY ENFORCEMENT FOR MATRIX P
136 % Why: Due to fixed-point rounding and truncation on the FPGA, the 2x2 P matrix
137 % can lose mathematical symmetry. Making P(1,2) equal to P(2,1) ensures
138 % matrix stability and prevents the Kalman algorithm from diverging.
139 P12 = fi((P(1,2) + P(2,1)) / 2, T, F);
140 P(1,2) = P12;
141 P(2,1) = P12;
142
143
144 % STEP 12: POSITIVE-DEFINITENESS SAFEGUARDS
145 % Why: In theory, covariance values (diagonal of P) can never be negative
146 % or zero. FPGA round-off errors could clamp them below zero, causing model crashes.
147 % Clamping them to a tiny positive number (0.0001) guarantees numerical stability.
148 if P(1,1) < fi(0.0001, T, F)
149     P(1,1) = fi(0.0001, T, F);
150 end
151
152 if P(2,2) < fi(0.0001, T, F)
153     P(2,2) = fi(0.0001, T, F);
154 end
155
156
157 % STEP 13: EXTRACT SPEED OUTPUT
158 % Why: The first element of the state vector (x_est) is the filtered angular
159 % speed (w_est). We route this out of the function.
160 w_est = x_est(1);
161
162 end

```

Figure 1.8

This figure 1.8 shows the Kalman filter code that is written inside a MATLAB Function block. The code uses the motors state-space model to do two things. First it does the prediction step. Then it does the measurement correction step. The Kalman filter does not just use the encoder data. It looks at the encoder measurement and the prediction from the motor model. This helps get rid of changes in the measurement. It also helps get a stable speed estimate.

The Kalman gain, in the code is very important. It decides how much the system uses the measurement data. If the measurement data is good the filter uses the measurement more. If the measurement is not good it uses the motor model more. This is why the filter gives an estimate that uses both the model and the measurement. The Kalman filter code was changed to VHDL format using HDL Coder. Now it can run on an FPGA.

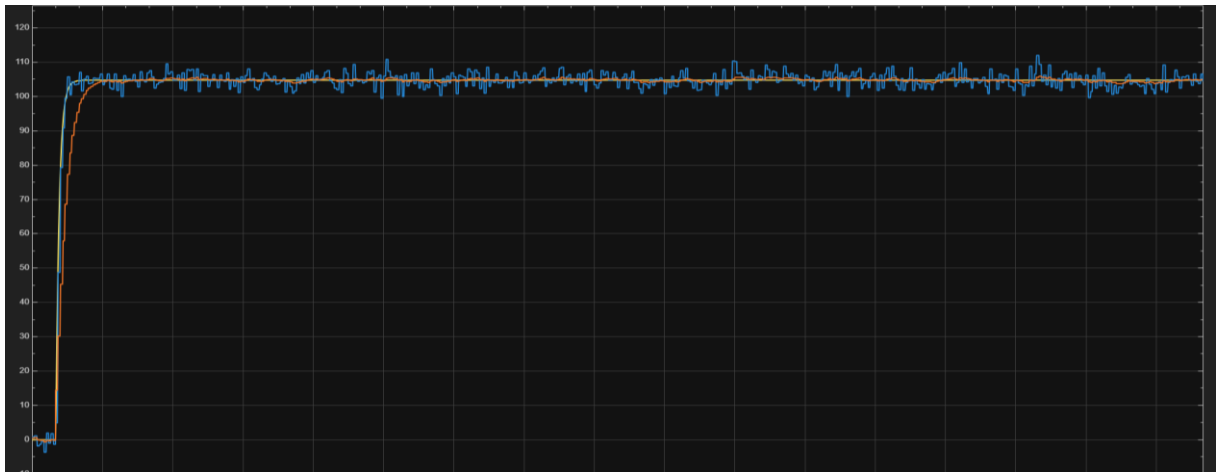


Figure 1.9

Figure 1.9 shows a full-length MATLAB/Simulink simulation of the adaptive Kalman filter driving the Simscape DC motor model from a stationary state to its nominal operating point of 12 V. The yellow line represents the actual, noise-free angular velocity generated internally by the Simscape model; this signal exists only in the simulation and was added solely for verification purposes, as there is no equivalent noise-free reference in the physical hardware. The blue line is a simulated encoder measurement obtained by adding band-limited white noise and quantization to the actual speed. The orange line is the output of the adaptive Kalman filter under test.

4.2 VIVADO DESIGN

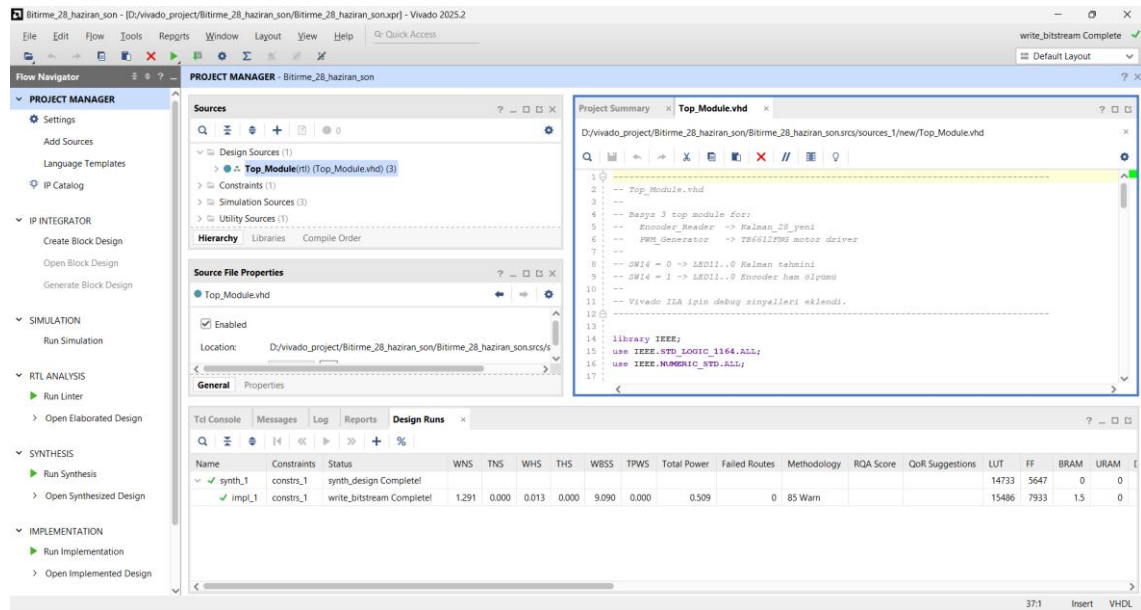


Figure 2.1

Figure 2.1 shows the top-level Vivado project view after all design resources have been added. The subsystem block containing the Kalman filter algorithm shown in Figure 1.6 was first converted to synthesizable VHDL code using the HDL Coder workflow advisor, resulting in four files: `MATLAB_Function.vhd`, which contains the fixed-point arithmetic data path; `Kalman_28_yeni.vhd`, which serves as the high-level wrapper for the filter core; `Kalman_28_yeni_pkg.vhd`, which defines common fixed-point data types; and `Kalman_28_yeni_tc.vhd`, a clock-enable timing controller that limits the filter's register updates to a 100 Hz rate derived from the 100 MHz FPGA system clock.



Figure 2.2

Figure 2.2 shows the FPGA resource usage reported by Vivado. The design utilizes LUTs, flip-flops, I/O ports, and other necessary FPGA resources. While the PWM generator and encoder reader modules consume fewer resources, the Kalman filter uses more LUTs and registers because it involves mathematical operations. This is an expected result because the Kalman filter is a more complex algorithm that involves matrix operations, multiplication, addition, and delay elements.

The resource usage report is essential to ensure the feasibility of the design in the Basys 3 board. When our resource usage does not exceed the capacity of the Basys 3 board we can be sure that our design is feasible for the Basys 3.

In addition, the resource usage report will allow us to improve our design in the future. We will be able to change some parameters like fixed-point bit widths or sampling time to use resources in the Basys 3. Even more we will be able to optimize our algorithm structure to fit resources in the Basys 3.

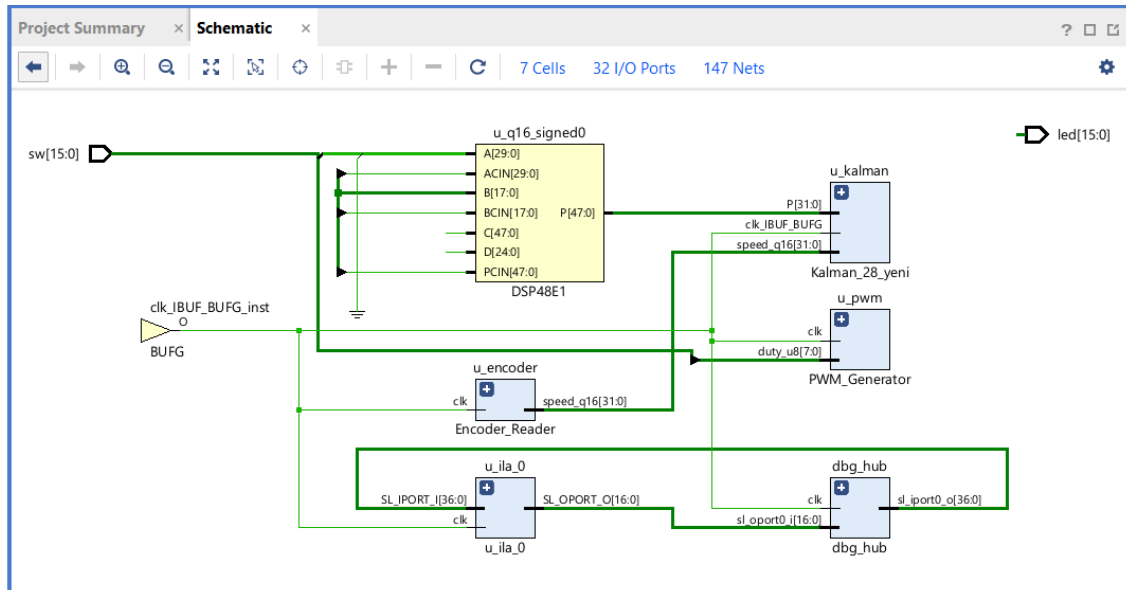


Figure 2.3

Figure 2.3 represents how data flow is implemented in the final version of our project in Vivado. The main blocks in our design are encoder reading, generating PWM signal, using a Kalman filter and debug blocks. All of them are included in the Top_Module.

The Encoder_Reader block is the module that reads A-B encoder signal that is provided by the motor and calculates speed in rad/s. This information will be used in the Kalman filter. Speed value can be analyzed with the help of Vivado ILA if any debugging is needed.

PWM_Generator block is the generator of PWM signal which controls the motor driver. Duty cycle value is taken from the switches on the FPGA. The Kalman_28_yeni block is the Kalman filter that was implemented by means of MATLAB and Simulink and then translated to VHDL code. This block uses speed information provided by the encoder and additional information from the motor model and gives estimated speed.

The u_ila_0 and dbg_hub blocks are the debug blocks that were added with the help of Vivado Hardware Manager. They allow analyzing behavior of different signals in the FPGA during time from the computer. Some of the signals are enc_rad_s_dbg, kalman_rad_s_dbg and duty_dbg.

This means we can check if the system is working properly by looking at the LED outputs on the FPGA or, by using the Vivado ILA.

5. FINDINGS AND CONCLUSIONS

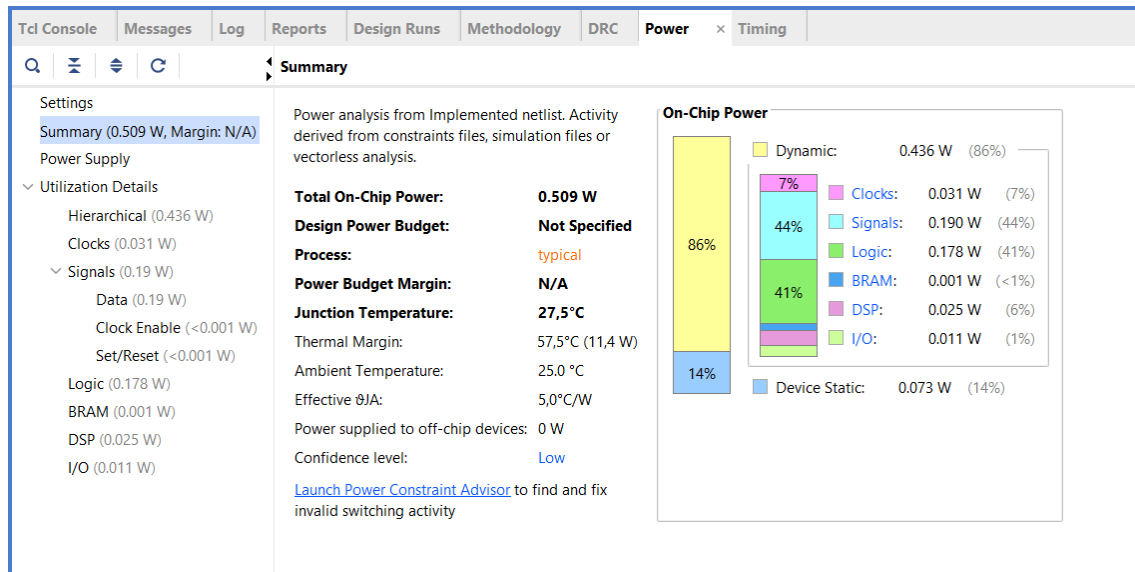


Figure 2.4

This figure 2.4 shows the power consumption data that Vivado calculated after the implementation process. The power analysis helps to find out how dynamic and static power a design uses when it runs on an FPGA. Dynamic power comes from the signals and clock activity that change during the design operation. Static power is the power that the FPGA uses all the time.

In this project PWM generation, encoder reading and Kalman filter calculations use the power. These components are the reasons for power usage.

The power consumption data is very important, for the project. It helps to understand how power the design uses. The FPGAs power usage is affected by the design operation. The dynamic and static power consumption are two parts of the total power usage.

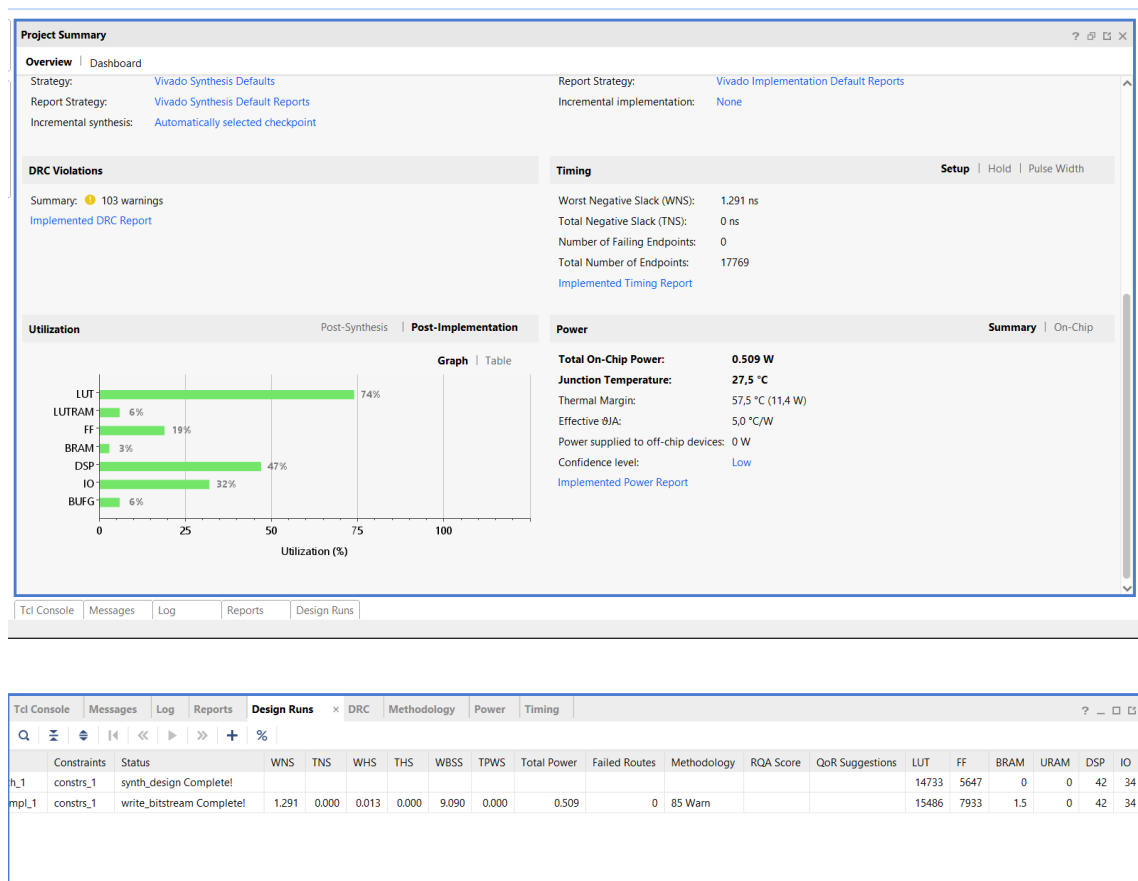


Figure 2.5

This figure 2.5 shows what the FPGA is using as reported by Vivado. The design is using things like LUTs and flip-flops and I/O ports and other things that the FPGA needs. The PWM generator and encoder reader parts are not using a lot of resources.. The Kalman filter is using more LUTs and registers. This is because the Kalman filter has to do a lot of math.

The Kalman filter is an algorithm. It has to do things like multiply and add numbers and work with matrices. It also has to remember some things, which's why it uses more resources. This makes sense because the Kalman filter is doing a lot of work.

It is good to know what resources the design is using. This helps us figure out if the design will work on the Basys 3 board. Since the design is not using many resources we know it will work on the Basys 3. This report can also help us make the design better. For example we can change things like the fixed-point bit widths or the sampling time or the way the algorithm works to use resources. The FPGA resource usage report is important, for this. We can use it to guide our efforts to optimize the design and use FPGA resources.

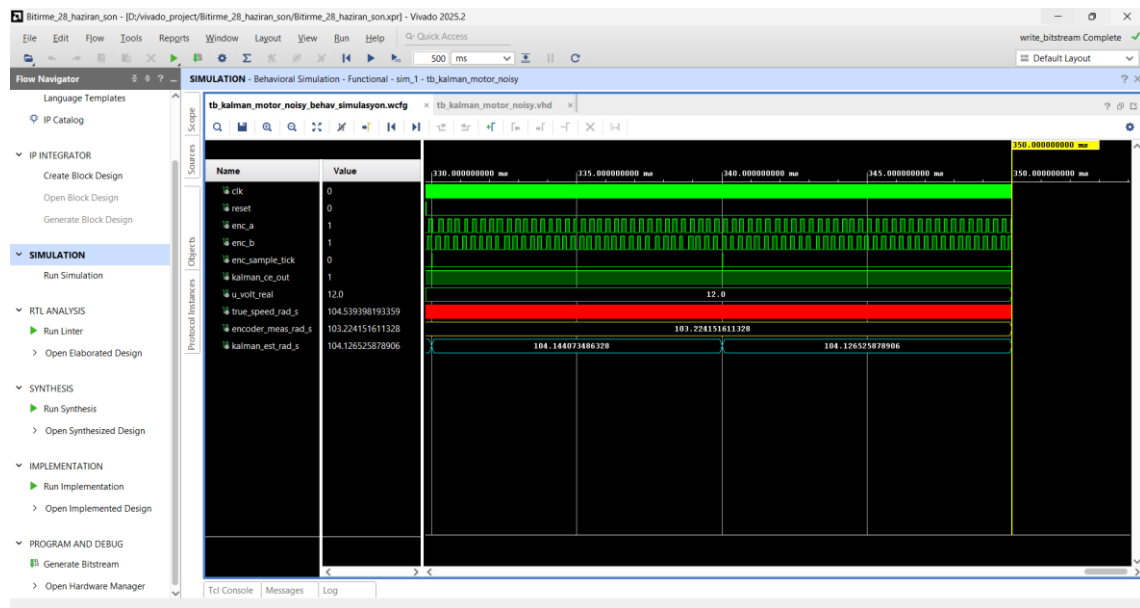


Figure 2.6

The aim of carrying out the simulation is to ensure that the VHDL code runs as desired before proceeding to implement the circuit in the hardware. In this way, we checked the encoder speed information, the PWM control, and the Kalman filter output to see how the process will work using digital logic.

It can be seen that there is a good match between the results of Vivado simulation and those obtained using the hardware as an indication of success in our project. Comparison was made between the speed response of the simulation and the values obtained using the Basys 3 board through LED and ILA outputs. In a system there might be small differences because of the motors mechanics, how well the encoder works and the driver circuit.

The fact that our simulation and hardware results are consistent for the same speed range shows that our design is working correctly. The simulation and actual results match closely which is good, for the VHDL code and the FPGA design.

Kalman: $3 \text{ rad/s} = 28.6 \text{ rpm}$

Enkoder: $2 \text{ rad/s} = 19 \text{ rpm}$

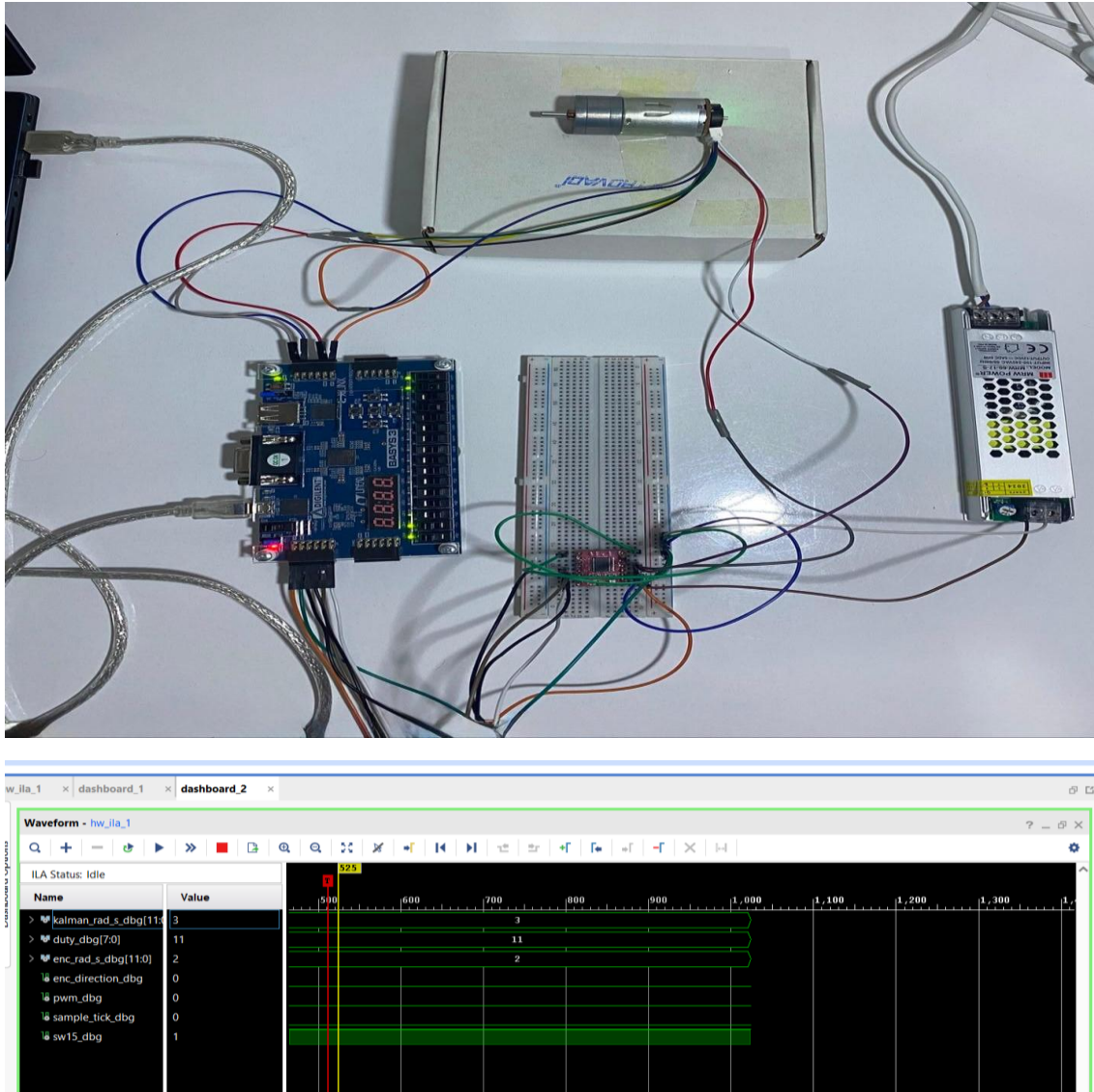


Figure 2.7

These images show the motor operating at a speed of approximately 3 rad/s in the low-speed range. The value obtained from the Vivado ILA screen and the LED output on the FPGA confirm the same speed level. When the 3 rad/s value is calculated in RPM, it equals $3 \times 9.549 = 28.6 \text{ RPM}$ ($\frac{\text{rad}}{\text{s}} \times \frac{60}{2\pi} = \text{rpm}$). This result shows that the motor operates at low speeds with low duty cycles and that the system can detect low speeds as well. The LED calculation is performed using binary logic. The number 3 is represented in binary as $2 + 1 = 3 \text{ rad/s}$, meaning that LED1 and LED0 are both lit. Therefore, the LED output on the FPGA physically represents the value of 3 rad/s. The fact that the same value is observed in Vivado proves that the LED output and the ILA debug data are in agreement.

Kalman: 52 rad/s = 497 rpm

Enkoder: 51 rad/s = 487 rpm

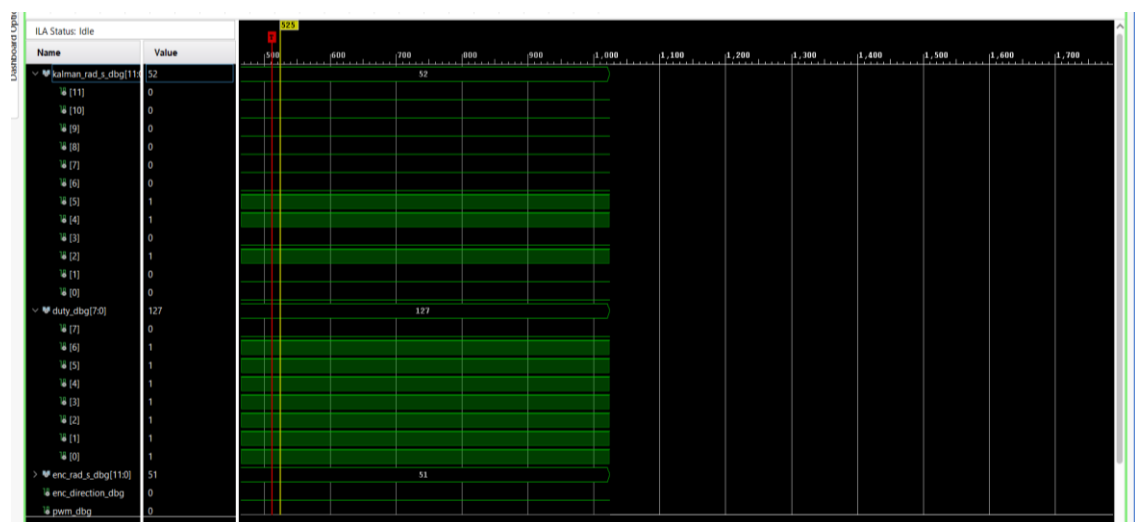
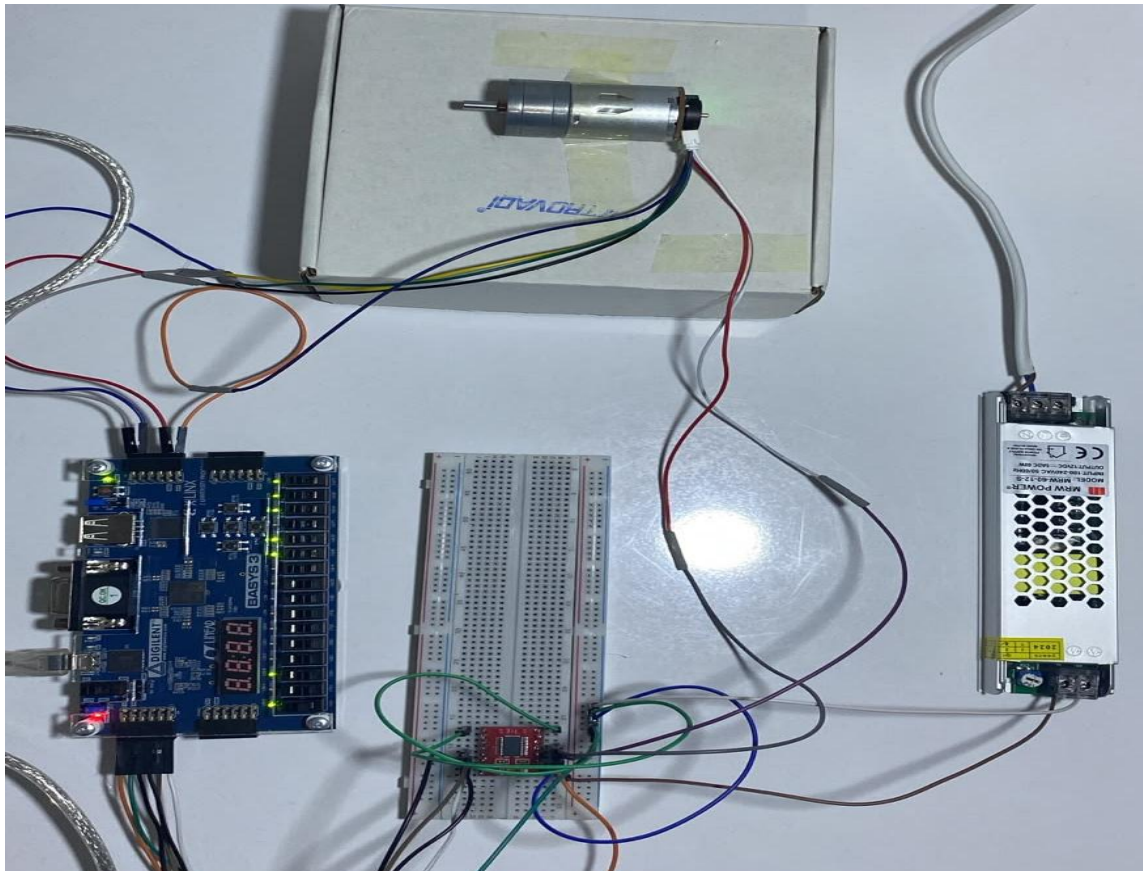


Figure 2.8

Kalman: 85 rad/s = 812 rpm

Enkoder: 87 rad/s = 831 rpm

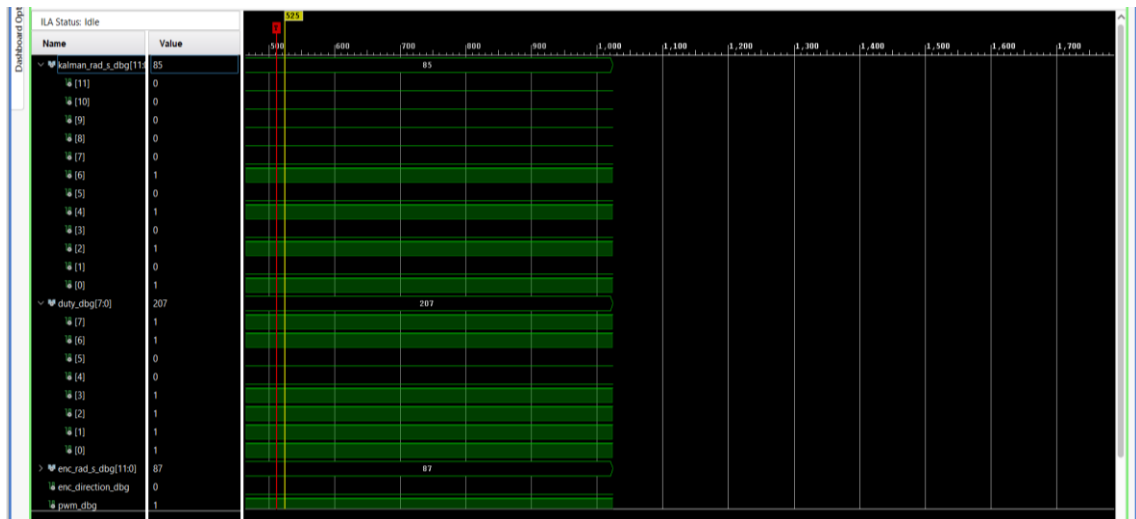
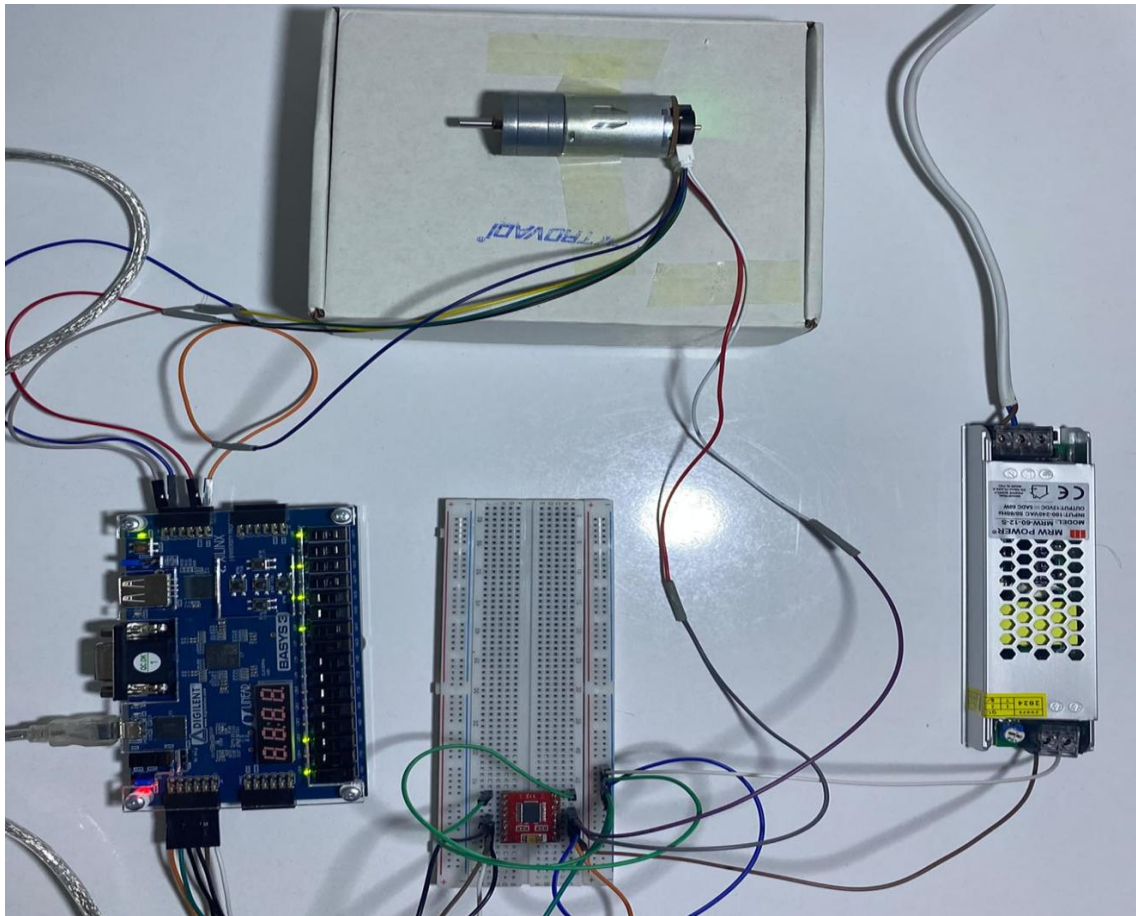


Figure 2.9

Kalman: 106 rad/s = 1012 rpm

Enkoder: 108 rad/s = 1031 rpm

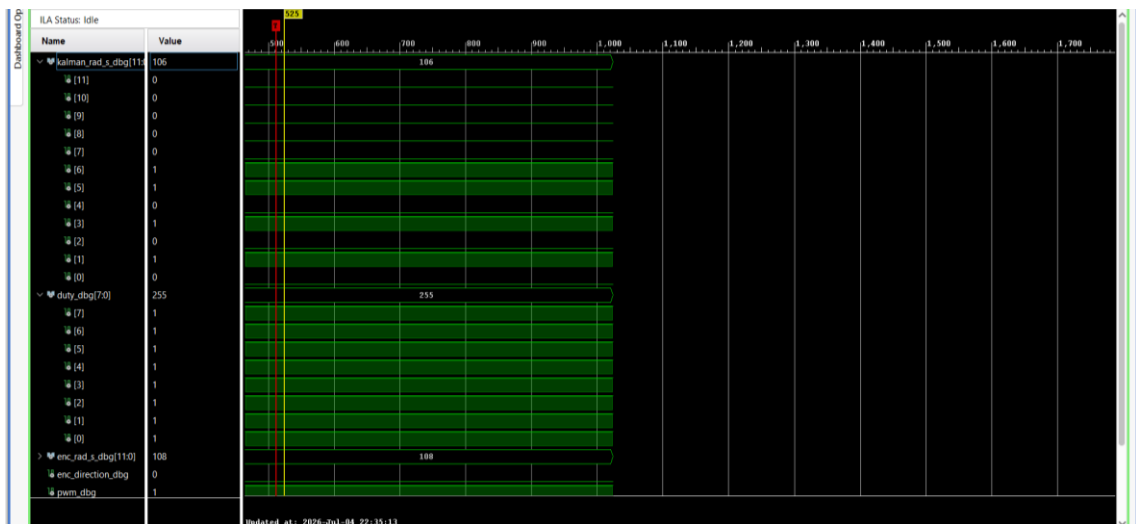
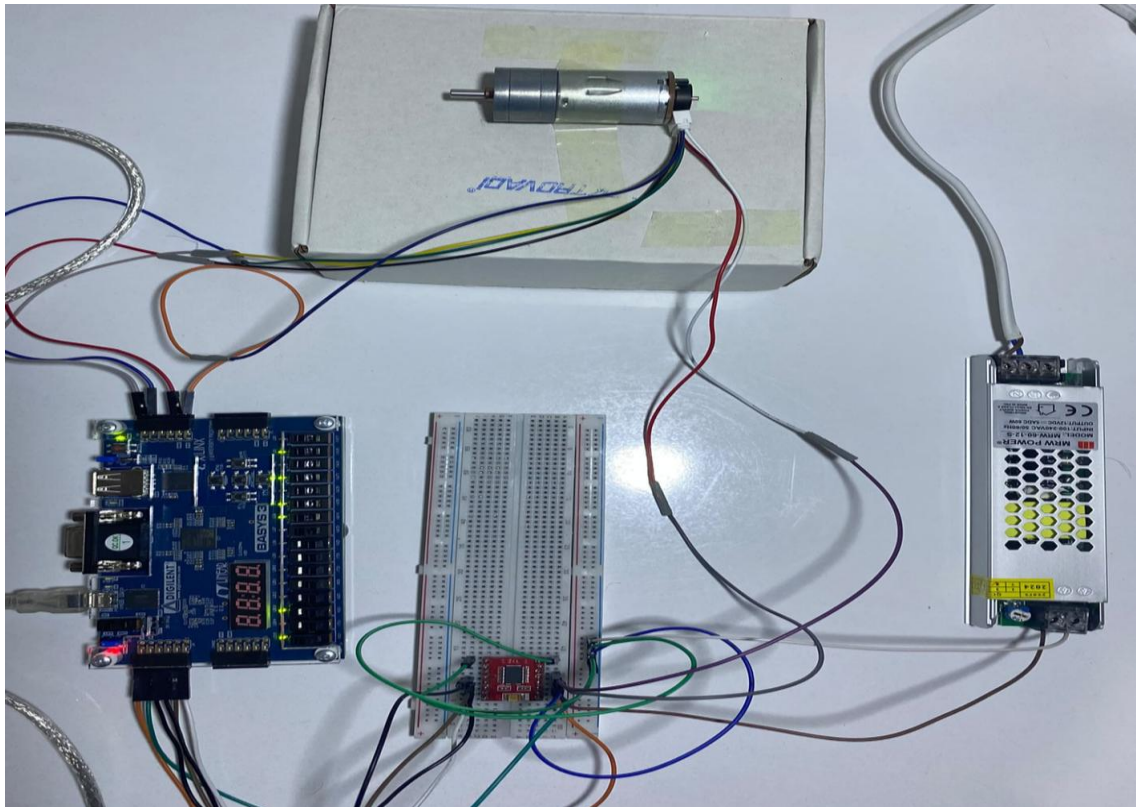


Figure 2.10

In conclusion that uses a Field Programmable Gate Array or FPGA to figure out how fast a DC motor is going was. It works. This system was made using a 12 V, 1000 RPM DC motor with an encoder a TB6612FNG motor driver and a Digilent Basys 3 FPGA development board. The FPGA was set up to make PWM signals read the signals from the encoder calculate the speed in rad/s and run the Kalman filter algorithm. The Kalman filter was designed using MATLAB and Simulink. Then it was converted to VHDL code and added to the Vivado project.

The results of the experiments show that the system can track the motor speed when the duty cycle is changed. When the motor is going slow it gets speeds of 3 rad/s and 11 rad/s. When it is going medium speed it gets speeds of about 52 rad/s and 85 rad/s. When the motor is going speed the duty cycle is 255 and the speed is about 105 rad/s. If we change this to RPM it is 1000 RPM, which is what the motor is supposed to do. This shows that the numbers used to calculate the speed from the encoder and the way the speed is measured on the FPGA are correct.

One of the things we found out from this project is that the lights on the FPGA are the same as the numbers we get from Vivado ILA. The lights show the speed in binary. We can also see the same speed numbers on the Vivado ILA screen. For example when the motor is going speed the encoder and Kalman filter both say it is going about 105 rad/s. This shows that the system works correctly on the board and on the Vivado debug screen. This means the project is not just theoretical. It also works on real hardware.

The Kalman filter is important because it helps make the speed estimate more stable. It does this by looking at the encoder measurements and the motor model at the time. The encoder measurements can be a little off because of the time it takes to sample them the resolution of the pulses, vibrations and the PWM effects. Of just using these measurements the Kalman filter combines them with the system model to get a better estimate. This means the motor speed is estimated using the model than just the encoder measurements.

So we were able to build a system that measures the DC motor speed in time on the FPGA and estimates it using a Kalman filter. When we look at the MATLAB and Simulink simulations the Vivado synthesis and implementation the lights on the FPGA and the Vivado ILA debug data it is clear that the system works. This project shows that it is possible to use encoder feedback and Kalman filtering in FPGA based real time control systems. In the future we could try to see how well the Kalman filter works when there are disturbances or we could look at the speed data in detail using MATLAB and Vivado.

3.APPENDICES

Top_Module.vhd file:

```
-----  
-- Top_Module.vhd  
--  
-- Basys 3 top module for:  
-- Encoder_Reader -> Kalman_28_yeni  
-- PWM_Generator -> TB6612FNG motor driver  
--  
-- SW14 = 0 -> LED11..0 Kalman tahmini  
-- SW14 = 1 -> LED11..0 Encoder ham ölçümü  
--  
-- Vivado ILA için debug sinyalleri eklendi.  
-----  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.NUMERIC_STD.ALL;  
  
entity Top_Module is  
    port (  
        clk      : in  std_logic; -- Basys 3 100 MHz clock  
        reset_btn : in  std_logic; -- BTNC, active-high reset  
  
        sw       : in  std_logic_vector(15 downto 0);  
  
        enc_a    : in  std_logic;  
        enc_b    : in  std_logic;  
  
        tb_pwma  : out std_logic;  
        tb_ain1  : out std_logic;  
        tb_ain2  : out std_logic;
```

```

tb_stby : out std_logic;

    led    : out std_logic_vector(15 downto 0)
);
end Top_Module;

```

architecture rtl of Top_Module is

```

-----

-- Voltage command scaling
--
--  $12\text{ V Q16} = 12 * 65536 = 786432$ 
--  $786432 / 255 = 3084$ 
-----

constant VOLT_PER_DUTY_Q16 : integer := 3084;

-----

-- Encoder speed scaling
--
-- CPR = 11
-- Gear ratio = 10
-- x4 quadrature decoding
--  $\text{COUNTS\_PER\_REV} = 11 * 10 * 4 = 440$ 
--
--  $T_s = 0.01\text{ s}$ 
--  $\text{SPEED\_GAIN\_Q16} = 93585$ 
-----

constant ENC_SPEED_GAIN_Q16 : integer := 93585;

signal reset          : std_logic;
signal duty_cmd       : std_logic_vector(7 downto 0);
signal direction_cmd  : std_logic;

```

```

signal pwm_sig      : std_logic;

signal u_q16_signed  : signed(31 downto 0);
signal data_q16      : std_logic_vector(31 downto 0);
signal w_est_q16     : std_logic_vector(31 downto 0);

signal enc_direction : std_logic;
signal enc_sample_tick : std_logic;

signal kalman_ce_out  : std_logic;

-----
-- ILA debug signals
--
-- Bunlar motor kontrolünü veya LED davranışını değiştirmez.
-- Sadece Vivado ILA üzerinden iç sinyalleri izlemek içindir.
-----

signal enc_rad_s_dbg    : std_logic_vector(11 downto 0);
signal kalman_rad_s_dbg : std_logic_vector(11 downto 0);
signal selected_rad_s_dbg : std_logic_vector(11 downto 0);

signal duty_dbg          : std_logic_vector(7 downto 0);
signal sw14_dbg          : std_logic;
signal sw15_dbg          : std_logic;
signal sample_tick_dbg   : std_logic;
signal pwm_dbg           : std_logic;
signal enc_direction_dbg : std_logic;

attribute mark_debug : string;
attribute mark_debug of enc_rad_s_dbg    : signal is "true";
attribute mark_debug of kalman_rad_s_dbg : signal is "true";
attribute mark_debug of selected_rad_s_dbg : signal is "true";
attribute mark_debug of duty_dbg          : signal is "true";

```

```

attribute mark_debug of sw14_dbg      : signal is "true";
attribute mark_debug of sw15_dbg      : signal is "true";

attribute mark_debug of sample_tick_dbg : signal is "true";
attribute mark_debug of pwm_dbg        : signal is "true";
attribute mark_debug of enc_direction_dbg : signal is "true";

begin

reset      <= reset_btn;
duty_cmd   <= sw(7 downto 0);
direction_cmd <= sw(15);

-----
-- ILA debug assignments
--
-- enc_rad_s_dbg    = encoder rad/s integer
-- kalman_rad_s_dbg = Kalman rad/s integer
-- selected_rad_s_dbg = LED11..0 üzerinde gösterilen değer
-----

enc_rad_s_dbg    <= data_q16(27 downto 16);
kalman_rad_s_dbg <= w_est_q16(27 downto 16);

selected_rad_s_dbg <= data_q16(27 downto 16) when sw(14) = '1'
                    else w_est_q16(27 downto 16);

duty_dbg      <= duty_cmd;
sw14_dbg      <= sw(14);
sw15_dbg      <= sw(15);
sample_tick_dbg <= enc_sample_tick;
pwm_dbg       <= pwm_sig;
enc_direction_dbg <= enc_direction;

-----

```

```

-- PWM generator
-----

u_pwm : entity work.PWM_Generator(rtl)

generic map (
    CLK_HZ => 1000000000,
    PWM_HZ => 20000
)
port map (
    clk    => clk,
    reset  => reset,
    duty_u8 => duty_cmd,
    pwm_out => pwm_sig
);

tb_pwma <= pwm_sig;

-----

-- TB6612FNG direction and standby control
--
-- direction_cmd = 0:
--  AIN1 = 1, AIN2 = 0
--
-- direction_cmd = 1:
--  AIN1 = 0, AIN2 = 1
-----

tb_ain1 <= '1' when (reset = '0' and direction_cmd = '0') else '0';
tb_ain2 <= '1' when (reset = '0' and direction_cmd = '1') else '0';

-- STBY high olursa motor sürücü aktif olur
tb_stby <= '1' when reset = '0' else '0';

-----

-- Encoder reader

```

```

u_encoder : entity work.Encoder_Reader(rtl)
    generic map (
        SAMPLE_TICKS => 1000000,

        SPEED_GAIN_Q16 => ENC_SPEED_GAIN_Q16
    )
    port map (
        clk      => clk,
        reset    => reset,
        enc_a     => enc_a,
        enc_b     => enc_b,
        speed_q16 => data_q16,
        direction => enc_direction,
        sample_tick => enc_sample_tick
    );

-----
-- Convert PWM duty to Kalman input voltage u in Q16.
--
-- Bu kısım senin çalışan versiyonundaki gibi bırakıldı.
-----

process(duty_cmd, direction_cmd)
    variable mag_int : integer;
    variable mag_s   : signed(31 downto 0);
begin
    mag_int := to_integer(unsigned(duty_cmd)) * VOLT_PER_DUTY_Q16;
    mag_s   := to_signed(mag_int, 32);

    -- Encoder pozitif okuduğu yönde Kalman'a da pozitif u veriyoruz.
    if direction_cmd = '1' then
        u_q16_signed <= mag_s;
    else
        u_q16_signed <= -mag_s;
    end if;
end process;

```

```

end process;

-- HDL Coder Kalman core
-- Kalman input:
-- u   = PWM duty'den hesaplanan voltage command
-- data = ham encoder hız ölçümü
-----

u_kalman : entity work.Kalman_28_yeni(rtl)
  port map (
    clk      => clk,
    reset    => reset,
    clk_enable => '1',
    u        => std_logic_vector(u_q16_signed),
    data     => data_q16,
    ce_out   => kalman_ce_out,
    w_est    => w_est_q16
  );
-- LED debug
--
-- SW14 = 0 -> Kalman tahmini
-- SW14 = 1 -> Encoder ham hızı
--
-- LED11..0 rad/s integer kısmını gösterir.
-- RPM = LED_value * 9.549
-----

led(11 downto 0) <= data_q16(27 downto 16) when sw(14) = '1'
                  else w_est_q16(27 downto 16);

led(12)    <= enc_sample_tick;
led(13)    <= pwm_sig;
led(14)    <= sw(14);      -- SW14 göstergesi
led(15)    <= direction_cmd; -- SW15 yön göstergesi

end rtl;

```

5. REFERENCES

- [1] Welch, G., & Bishop, G. (2006). *An introduction to the Kalman filter*. University of North Carolina.
- [2] Stengel, R. F. (1994). *Optimal control and estimation*. Dover Publications.
- [3] Ogata, K. (2010). *Modern control engineering*. Prentice Hall.
- [4] Simon, D. (2006). *Optimal state estimation: Kalman, H_∞ , and nonlinear approaches*. Wiley.
- [5] Jadan, H. I. V., Vera, C. R. M., Intriago, R. P., & Padilla, V. S. (2016). *Implementing a Kalman filter on FPGA embedded processor for speed control of a DC motor using low resolution incremental encoders*. In *Proceedings of the World Congress on Engineering and Computer Science (WCECS 2016)* (Vol. I, pp. 367–371). https://www.iaeng.org/publication/WCECS2016/WCECS2016_pp367-371.pdf
- [6] Kung, Y.-S., et al. (2013). *FPGA Realization of Sensorless PMSM Speed Controller Based on Extended Kalman Filter*
https://www.researchgate.net/publication/270675091_FPGA_Realization_of_Sensorless_PMSM_Speed_Controller_Based_on_Extended_Kalman_Filter
- [7] S. K. Ravindran, "Fixed-Point Kalman Filter on PYNQ-Z2 FPGA," *International Journal of Innovative Science and Research Technology*, vol. 10, no. 11, pp. 2818-2823, Nov. 2025. <https://ijisrt.com/assets/upload/files/IJISRT25NOV569.pdf>
- [8] MathWorks. (2023). *c2d—Convert model from continuous to discrete time* (MATLAB documentation). <https://www.mathworks.com>
- [9] Kruse, T., Griebel, T., & Graichen, K. (2025). *Adaptive Kalman filtering: Measurement and process noise covariance estimation using Kalman smoothing*. *IEEE Access*, 13, 11863–11875. <https://doi.org/10.1109/ACCESS.2025.3528348>
- [10] Abougarair, A. J. (n.d.). Enhancing speed estimation in DC motors using the Kalman filter method: A comprehensive analysis. Retrieved from: https://www.academia.edu/127119074/Enhancing_Speed_Estimation_in_DC_Motors_using_the_Kalman_Filter_Method_A_Comprehensive_Analysis

- [11] İnan, R., & Üzümlü, O. M. (2023). *Speed-sensorless DTC of BLDC motor with EKF-based estimator capable of load torque estimation for electric vehicle*. Dergipark. Retrieved from <https://dergipark.org.tr/tr/download/article-file/2712903>
- [12] Ding, J., Chen, L., Cao, Z., & Guo, H. (2021). *Convergence analysis of the modified adaptive extended Kalman filter for the parameter estimation of a brushless DC motor*. *International Journal of Robust and Nonlinear Control*, 31(16), 7606–7620. <https://doi.org/10.1002/rnc.5706>
- [13] Pasricha, R., & Sharma, S. (2009). *An FPGA-based design of fixed-point Kalman filter*. *ICGST International Journal on Digital Signal Processing (DSP)*, 9(1), 1–9. https://www.researchgate.net/profile/Ashraf-Aboshosha/publication/292971388_DSP-Volume9-Issue1_P1181606473/links/56b2ef0408ae795dd5c7d81d/DSP-Volume9-Issue1-P1181606473.pdf#page=5
- [14] J. P. Oliver, J. Curto, D. Bouvier, M. Ramos and E. Boemo, "Clock gating and clock enable for FPGA power reduction," *2012 VIII Southern Conference on Programmable Logic, Bento Gonçalves, Brazil, 2012*, pp. 1-5, [doi: 10.1109/SPL.2012.6211782](https://doi.org/10.1109/SPL.2012.6211782).